Applied Software Project Report

By

Syed Abbas Raza

A Master's Project Report submitted to Scaler Neovarsity - Woolf
in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

May, 2025

Scaler Mentee Email ID : abbas.sd17@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 29/03/2026

Certification

I confirm that I have overseen and reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

Declaration

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 4th April 2025 to 28th April 2025, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct.

Syed Abbas Raza

Date: 29 Mar 2026

Acknowledgment

I would like to express my sincere gratitude to my supervisor Naman Bhalla and all the instructors at Scaler Neovarsity for their invaluable guidance and support throughout this project. Their practical teaching approach, which combined theoretical foundations with hands-on implementation, gave me the skills and confidence to build a production-ready backend system.

I am deeply thankful to the Scaler Academy team for designing a curriculum that bridges the gap between academia and industry. The lecture-by-lecture progression — from Django REST Framework fundamentals all the way to cloud deployment and payment gateway integration — has been an exceptional learning journey.

Finally, I am grateful to my family and friends whose constant encouragement and motivation have been instrumental in my completion of the Master of Science in Computer Science program. This achievement would not have been possible without their unwavering support.

Table of Contents

List of Tables..... 7

List of Figures..... 8

Abstract..... 9

Project Description..... 9

Requirement Gathering..... 11

Class Diagrams..... 15

Database Schema Design..... 17

Feature Development Process..... 22

Deployment Flow..... 27

Technologies Used..... 32

Conclusion..... 37

References..... 39

List of Tables

Table No.	Title	Page No.
1.1	Functional Requirements	11
1.2	Non-Functional Requirements	12
1.3	User Roles and Use Cases	13
1.4	Feature Set	14
2.1	Product Table Schema	17
2.2	Category Table Schema	18
2.3	Order Table Schema	19
2.4	Payment Table Schema	20
2.5	Foreign Key Relationships	21
3.1	API Endpoints — Products	23
3.2	API Endpoints — Payments	24
3.3	Cache Performance Benchmark	26
4.1	Technologies Used	32
4.2	AWS Services Used	33

List of Figures

Figure No.	Title	Page No.
1.1	High-Level Architecture Diagram	10
1.2	Project Development Flow	10
1.3	Use Case Diagram	13
2.1	Class Diagram — Products App	15
2.2	Class Diagram — Payments App	16
2.3	Entity Relationship Diagram (ERD)	21
3.1	Payment Flow — End-to-End	22
3.2	Redis Cache Flow Diagram	25
3.3	Pagination Flow Diagram	26
4.1	AWS Deployment Architecture	27
4.2	VPC and Security Group Layout	29
4.3	Elastic Beanstalk Deployment Flow	30
4.4	Request Flow: Browser to Django	31
A1	GET /api/products/ — product list	
A2	GET /api/products/filter/ — price range filter	
A3	Django administration — login.	
A4	Django administration — registered models	
A5	GET /api/products/1/stock/ — stock check	
A6	GET /api/payments/ — payment list with idempotency & webhook fields	
A7	GET /api/products/health/ — health check	
A8	GET /api/products/search/ returns 405 — search endpoint is POST-only	
A9	POST /api/products/search/ — pagination & search results	
A10	GET /api/products/1/ — product detail with Redis	

	caching	

Applied Software Project

Abstract

This report documents the design, development, and deployment of a production-ready E-Commerce Backend System built using Django REST Framework (DRF) as part of the Scaler Neovarsity Backend Specialization program. The project was developed incrementally across ten lectures, each introducing progressively advanced backend engineering concepts.

The system provides a comprehensive suite of RESTful APIs for product management, including create, read, update, and delete (CRUD) operations, advanced filtering using Django's Q objects, full-text search with server-side pagination, and Redis-based response caching to significantly improve API performance. The backend implements Django ORM relationships including One-to-One, One-to-Many, and Many-to-Many associations between core entities such as Products, Categories, Orders, and User Profiles.

A key feature of the project is its integrated payment processing module, which uses Razorpay as the payment gateway. The payment system is designed with idempotency at its core, ensuring that duplicate payment requests do not result in duplicate charges. The system supports partial payments, webhook-based payment confirmation, and signature verification for security. The payment gateway is abstracted using Python's Abstract Base Class (ABC) pattern, allowing future gateways such as Stripe or PayU to be integrated with minimal code changes.

Cross-cutting concerns such as request logging, exception handling, and access control are handled via custom Django middlewares and decorators. The project follows clean architecture principles with a clear separation between the view layer, service layer, and data access layer. For cloud deployment, the application is configured for AWS using Elastic Beanstalk (EBS), with Amazon RDS (MySQL) as the production database. The deployment architecture leverages VPC, Security Groups, Route 53, and a Load Balancer for scalability and security.

Project Description

The E-Commerce Backend is a RESTful web service built to power an online retail platform. It serves as the backend engine that handles product listings, category management, order creation, and payment processing. The system is built entirely with Python and Django, following the Model-View-Serializer (MVS) architectural pattern introduced by Django REST Framework.

Project Objectives

- Build a scalable and maintainable RESTful API backend using Django REST Framework.
- Implement clean database design with proper ORM relationships and model inheritance.
- Apply advanced Django ORM techniques including Q objects, query chaining, and N+1 prevention.
- Implement cross-cutting concerns (logging, exception handling) via middleware and decorators.
- Integrate Razorpay payment gateway with idempotency guarantees and webhook support.
- Deploy the application on AWS using Elastic Beanstalk, RDS, and VPC architecture.
- Apply caching with Redis to achieve measurable API response time improvements.

System Architecture Overview

The project follows a layered architecture. The Client (browser or mobile app) sends HTTP requests to the Django server. In development, Django's built-in server handles requests directly. In production on AWS, requests flow through Route 53 (DNS), a Load Balancer, a VPC-protected EC2 instance, Gunicorn (WSGI server), and finally reach the Django application layer.

Within Django, each request is processed by the middleware stack (LoggerMiddleware), routed to the appropriate view function, which delegates business logic to service classes, queries the database through the ORM, and returns a serialized JSON response.

High-Level Architecture:

```
Client (Browser / Mobile App)
       ↓
  Route 53 (DNS Resolution)
       ↓
  Load Balancer (Traffic Distribution)
       ↓
  VPC → Security Groups → EC2 Instance
       ↓
  Gunicorn (WSGI Server)
       ↓
  Django Application
   ├── Middleware (LoggerMiddleware)
   ├── URL Router
   ├── Views (API Endpoints)
   ├── Serializers (Data Validation)
   ├── Services (Business Logic)
   ├── ORM (Database Access)
   └── Cache (Redis)
       ↓
  AWS RDS (MySQL Production Database)
```

Figure 1.1: High-Level Architecture Diagram

Project Modules

The project is organized into two primary Django applications:

- Products App: Handles all product-related functionality including CRUD operations, category management, advanced filtering, search with pagination, stock checking, and Redis caching.
- Payments App: Handles payment initiation, verification, webhook processing, and gateway abstraction using abstract base classes. Supports Razorpay integration with idempotency.

Requirement Gathering

Functional Requirements

The following table lists the functional requirements of the E-Commerce Backend system, derived from the lecture notes and project scope.

Table 1.1: Functional Requirements

Req. ID	Requirement Description	Priority
FR-01	System shall provide REST APIs for Product CRUD operations	High
FR-02	System shall support category-based product organization	High
FR-03	System shall support filtering products by price, availability, category	High
FR-04	System shall provide paginated search results for products	High
FR-05	System shall cache product responses using Redis	Medium
FR-06	System shall support order creation with multiple products	High
FR-07	System shall initiate payments via Razorpay gateway	High
FR-08	System shall verify payment using signature validation	High
FR-09	System shall handle idempotent payment requests	High
FR-10	System shall receive and process payment webhooks	High
FR-11	System shall log every API request with path and status code	Medium
FR-12	System shall raise and handle custom exceptions with correct HTTP status	Medium
FR-13	System shall provide a health check endpoint	Low

Non-Functional Requirements

Table 1.2: Non-Functional Requirements

Req. ID	Requirement Description	Category
NFR-01	API response time for cached endpoints shall be under 10ms	Performance
NFR-02	System shall handle duplicate payment requests without double-charging	Reliability
NFR-03	All sensitive credentials shall be stored as environment variables	Security
NFR-04	Payment verification shall use HMAC-SHA256 signature validation	Security
NFR-05	System shall scale horizontally on AWS with Load Balancer	Scalability
NFR-06	Database shall be centralized (AWS RDS) for multi-instance consistency	Reliability
NFR-07	All API endpoints shall return appropriate HTTP status codes	Standards
NFR-08	Code shall follow Python PEP8 style guidelines	Maintainability
NFR-09	Each app shall have a clear separation of concerns (views/services)	Maintainability

Users and Use Cases

The E-Commerce Backend serves three primary user types: End Customers who browse and purchase products, Store Administrators who manage the product catalog, and Payment Gateway Services (Razorpay) which send webhook notifications.

Table 1.3: User Roles and Use Cases

Actor	Use Case	Description
Customer	Browse Products	View paginated list of available products with search and filters
Customer	View Product Details	Get detailed information about a specific product (cached)
Customer	Place Order	Create an order with multiple products
Customer	Make Payment	Initiate payment via Razorpay and receive payment link
Customer	Confirm Payment	System verifies payment after

		gateway redirect
Admin	Create Product	Add new products with name, price, category, availability
Admin	Update Product	Modify product details; cache automatically invalidated
Admin	Delete Product	Remove product from catalog
Admin	Manage Categories	Create and list product categories
Admin	View Payments	Monitor all payment transactions and statuses
Razorpay	Send Webhook	Notify backend of payment capture/failure events
Razorpay	Payment Callback	Redirect user to callback URL after payment

Feature Set

Table 1.4: Feature Set

#	Feature	Module	Status	Lecture
1	Product CRUD APIs	Products	Implemented	1, 3
2	Product Serialization (DRF)	Products	Implemented	1
3	Abstract Base Model (AuditData)	Products	Implemented	2
4	Q Objects & Query Chaining	Products	Implemented	2
5	ORM Relationships (FK, M2M, O2O)	Products	Implemented	3
6	N+1 Problem Fix (select_related)	Products	Implemented	3
7	Custom Exception Handling	Products	Implemented	4
8	Python Decorators	Products	Implemented	4
9	Logger Middleware	Products	Implemented	5
10	Redis Caching (GET product)	Products	Implemented	10
11	Cache Invalidation on Update	Products	Implemented	10
12	Pagination (PageNumberPagination)	Products	Implemented	10
13	POST-based Search API	Products	Implemented	10
14	Idempotent Payment Model	Payments	Implemented	8
15	Webhook Handler	Payments	Implemented	8, 9
16	Abstract Payment Gateway	Payments	Implemented	9
17	Razorpay Integration	Payments	Implemented	9
18	Payment Service Layer	Payments	Implemented	9

19	Signature Verification	Payments	Implemented	8, 9
20	AWS RDS Config	Deployment	Configured	6
21	Elastic Beanstalk Config	Deployment	Configured	7, 8

Class Diagrams

The following section describes the Low Level Design (LLD) of the project through class diagrams and their relationships. The system is composed of two main Django applications: Products and Payments.

Products App — Class Structure

The Products app follows Django's Model-View pattern augmented with custom components. The core model hierarchy uses Python's abstract class mechanism (class Meta: abstract = True) for model inheritance, ensuring audit timestamps propagate to all inheriting models.

```
+------------------+
|   AuditData      |  <<abstract>>
|------------------|
| + created_at     |
| + updated_at     |
+--------+---------+
         |
    inherits
         |
   +-----+-------+         +------------+
   |   Product   |-------->|  Category  |
   |-------------|  FK     |------------|
   | + id        |         | + name(PK) |
   | + name      |         | + desc     |
   | + price     |         +------------+
   | + is_avail  |
   | + category  |<-------+----------+
   +------+------+  M2M   |   Order  |
          |               |----------|
          |               | + id     |
          |               | + user   |
          |               | + status |
          |               +----------+
   +-------+------+
   | ProductSerializer |
   | CategorySerializer|
   | OrderSerializer   |
+--------------------+
   | ProductPaginator  |
   | PageView (APIView)|
+--------------------+
```

Figure 2.1: Class Diagram — Products App

Payments App — Class Structure

The Payments app demonstrates the Abstract Factory pattern. PaymentGateway is an Abstract Base Class (ABC) from Python's abc module. RazorpayPaymentGateway implements the abstract interface. PaymentService uses dependency injection to accept any gateway implementation, making the system extensible to multiple payment providers.

```
+----------------------+
| PaymentGateway       |  <<ABC>>
|----------------------|
| + create_payment_link|  <<abstract>>
| + verify_payment     |  <<abstract>>
+----------+-----------+
           |
       implements
           |
+----------+-----------+
| RazorpayPaymentGateway|
|----------------------|
| - client: Razorpay   |
| + create_payment_link|
| + verify_payment     |
+----------+-----------+
           |
       injected into
           |
+----------+-----------+
| PaymentService       |
|----------------------|
| - gateway            |
| + initiate_payment   |
| + verify_and_confirm |
| + handle_webhook     |
+----------+-----------+
           |
       used by
           |
+----------+-----------+
| PaymentView (APIView)|
| WebhookView          |
| VerifyPaymentView    |
+----------------------+
```

Figure 2.2: Class Diagram — Payments App

Middleware & Decorator Class Structure

```
+---------------------+
| LoggerMiddleware    |
|---------------------|
| - get_response      |
| + __init__()        |
| + __call__(request) |
+---------------------+

+----------------------------+
| Decorators (functions)     |
|----------------------------|
| + after_10pm_deny(func)    |
| + greet_decorator(func)    |
| + log_request(func)        |
+----------------------------+

+----------------------------+
| Custom Exceptions          |
|----------------------------|
| ProductOutOfStockException |
| InvalidOrderException      |
+----------------------------+
```

Database Schema Design

The database schema is designed using Django's ORM. In development, SQLite is used. In production on AWS, the application connects to a MySQL 8.0 database hosted on Amazon RDS. The schema reflects the relationships taught across Lectures 2 and 3.

Tables

Table 2.1 — Product

Table 2.1: Product Table Schema

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique product identifier
name	VARCHAR(255)	NOT NULL	Product name
description	TEXT	NULL	Product description
price	FLOAT	NOT NULL	Product price in INR
is_available	BOOLEAN	DEFAULT TRUE	Stock availability flag
category_id	VARCHAR(100)	FK → Category(name), NULL	Product category
created_at	DATETIME	AUTO (on create)	Inherited from AuditData
updated_at	DATETIME	AUTO (on update)	Inherited from AuditData

Table 2.2 — Category

Table 2.2: Category Table Schema

Column	Type	Constraints	Description
name	VARCHAR(100)	PRIMARY KEY	Category name (custom PK)
description	TEXT	NULL	Category description

Table 2.3 — Order

Table 2.3: Order Table Schema

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Order ID
order_number	VARCHAR(100)	UNIQUE, NOT NULL	Human-readable order ref

user_id	INTEGER	FK → auth_user(id), CASCADE	Order owner
status	VARCHAR(20)	DEFAULT 'PENDING'	Order lifecycle status
total_amount	FLOAT	DEFAULT 0.0	Total order value
created_at	DATETIME	AUTO (on create)	From AuditData
updated_at	DATETIME	AUTO (on update)	From AuditData

Table 2.4 — Payment

Table 2.4: Payment Table Schema

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Internal payment record ID
payment_id	VARCHAR(255)	UNIQUE, NOT NULL	Idempotency key
order_id	VARCHAR(100)	NOT NULL	Reference to order
user_id	INTEGER	FK → auth_user(id)	Paying user
amount	INTEGER	NOT NULL	Amount in paise (e.g. 50000 = ₹500)
status	VARCHAR(20)	DEFAULT 'PENDING'	Payment status
payment_method	VARCHAR(20)	DEFAULT 'RAZORPAY'	Gateway used
gateway_payment_id	VARCHAR(255)	NULL	Razorpay payment ID
gateway_order_id	VARCHAR(255)	NULL	Razorpay order/link ID
gateway_signature	VARCHAR(255)	NULL	HMAC signature for verification
webhook_received	BOOLEAN	DEFAULT FALSE	Webhook confirmation flag
created_at	DATETIME	AUTO (on create)	Timestamp
updated_at	DATETIME	AUTO (on update)	Last update timestamp

Junction Table — Order_Products (M2M)

Column	Type	Constraints
order_id	INTEGER	FK → Order(id)
product_id	INTEGER	FK → Product(id)

Composite Primary Key: (order_id, product_id) — auto-created by Django for M2M fields.

Foreign Key Relationships

Table 2.5: Foreign Key Relationships

Relationship	Type	Description
Product.category_id → Category.name	Many-to-One	Many products can belong to one category
Order.user_id → User.id	Many-to-One	Many orders can belong to one user
Order.products → Product (M2M)	Many-to-Many	Each order contains multiple products
Profile.user_id → User.id	One-to-One	Each user has exactly one profile
Payment.user_id → User.id	Many-to-One	A user can have many payments

Cardinality of Relations

- Category → Products: 1:M (one category has many products)
- User → Orders: 1:M (one user places many orders)
- Order → Products: M:M (one order has many products, one product in many orders)
- User → Profile: 1:1 (one user has exactly one profile)
- User → Payments: 1:M (one user makes many payments)
- Order → Payments: 1:M (one order can have multiple payment attempts — partial payments)

Entity Relationship Diagram (ERD)

```
+----------+       +-----------+       +----------+
|  User    |1-----M|   Order   |M-----M|  Product |
+----------+       +-----------+       +----------+
|id        |       |id         |       |id        |
|username  |1---1  |order_number       |name      |
|email     |   |   |user_id(FK)|       |price     |
+----------+   |   |status     |       |category  |
            +------+          |       |is_avail  |
            |Profile|          |       +-----+----+
            |-------|          |             |
            |user_id|  +-------+-------+     M
            |phone  |  |   Payment     |     |
            |address|  |---------------|  +--+--------+
            +-------+  |payment_id(UQ) |  |  Category |
                       |order_id       |  |-----------|
                       |user_id(FK)    |  |name(PK)   |
                       |amount(paise)  |  |description|
```

```
|status         |  +-----------+
|gateway_pay_id |
|webhook_recv   |
+---------------+
```

Figure 2.3: Entity Relationship Diagram (ERD)

Feature Development Process

This section details the end-to-end development and performance optimization of the Payment Initiation feature — the most complex and business-critical feature of the project. This feature was developed across Lectures 8 and 9 and demonstrates the complete backend request flow from API call to database storage to gateway integration.

Selected Feature: Payment Initiation with Idempotency

The Payment Initiation feature allows a customer to pay for their order through Razorpay. It demonstrates several advanced backend concepts simultaneously: idempotency, abstract class design, service layer pattern, and payment gateway integration.

API Request Payload

Endpoint: POST /api/payments/initiate/

```
Request Headers:
  Content-Type: application/json
  Authorization: Bearer <token>

Request Body:
{
    "order_id": "ORD-2025-001",
    "amount": 49900
}

# amount is in PAISE (49900 paise = ₹499.00)
# Integer type avoids floating-point precision issues

Success Response (201 Created):
{
    "payment_id": "pay_a3f9c2e1b4d5...",
    "payment_link": "https://rzp.io/l/xYzAbc",
    "amount": 49900,
    "order_id": "ORD-2025-001"
}
```

MVC/MVS Architecture Flow

The request follows a strict separation of concerns across the Django application layers:

1. URL Router: Django matches POST /api/payments/initiate/ to PaymentView

2. Middleware (LoggerMiddleware): Logs: [MIDDLEWARE] ▶️ Request path: /api/payments/initiate/ | Method: POST

3. View (PaymentView): Validates request data using InitiatePaymentSerializer

4. Serializer: Validates: order_id (char, required), amount (int, min=1)

5. Service (PaymentService.initiate_payment): Generates unique payment_id (UUID-based idempotency key)

6. ORM (Payment.objects.create): Saves PENDING payment record to database

7. Gateway (RazorpayPaymentGateway.create_payment_link): Calls Razorpay API, receives short URL

8. Service: Updates Payment record with gateway_order_id

9. View: Returns 201 Created with payment_link and payment_id

10. Middleware: Logs: [MIDDLEWARE] ◀️ Response status: 201 | Duration: 342ms

Payment Flow Diagram

```
Customer App                Backend                  Razorpay
     |                        |                          |
     |-- POST /initiate/ ---->|                          |
     |                        |-- Validate request       |
     |                        |-- Generate payment_id    |
     |                        |-- Save PENDING to DB     |
     |                        |-- create_payment_link -->|
     |                        |<-- payment link ----------|
     |<-- 201 {payment_link} -|                          |
     |                        |                          |
     |-- Open payment_link -->|                          |
     |                        |                          |
     [User pays on Razorpay UI]                          |
     |                        |<-- Webhook: payment -----|
     |                        |    captured              |
     |                        |-- Verify signature       |
     |                        |-- Update Payment SUCCESS  |
     |<-- Callback redirect --|                          |
     |                        |                          |
```

Figure 3.1: Payment Flow — End-to-End

Idempotency Implementation

Idempotency ensures that multiple identical payment requests result in only one charge. This is critical in e-commerce to handle network failures, user double-clicks, and retry logic.

```
# Unique payment_id generated per payment ATTEMPT (not per order)
payment_id = f'pay_{uuid.uuid4().hex}'

# Saved to DB with UNIQUE constraint
payment = Payment.objects.create(
    payment_id=payment_id,  # Idempotency key
    order_id=order_id,
    user=user,
    amount=amount,
    status='PENDING'
)

# For partial payments: order_id is NOT the idempotency key
# Because one order_id can have MULTIPLE payment records
# e.g. Pay half by wallet, half by credit card
```

API Endpoints Summary

Table 3.1: API Endpoints — Products & Payments

Method	Endpoint	Description	Status Code
GET	/api/products/	List all products	200 OK
GET	/api/products/<id>/	Get product (Redis cached)	200 / 404
POST	/api/products/create/	Create product	201 Created
PUT	/api/products/<id>/update/	Update product + clear cache	200 OK
DELETE	/api/products/<id>/delete/	Delete product + clear cache	204 No Content
GET	/api/products/filter/	Filter with Q objects	200 OK
GET	/api/products/<id>/stock/	Check stock (custom exception)	200 / 409
POST	/api/products/search/	Search + paginate (Lecture 10)	200 OK
GET	/api/products/categories/	List categories	200 OK
GET	/api/products/health/	Health check	200 OK
POST	/api/payments/initiate/	Initiate Razorpay payment	201 Created
POST	/api/payments/verify/	Verify payment signature	200 / 400
POST	/api/payments/webhook/	Receive Razorpay webhook	200 OK
GET	/api/payments/	List all payments	200 OK

Redis Cache Performance

Redis caching was implemented for the GET /api/products/<id>/ endpoint. The cache stores serialized product data with a 300-second TTL. On cache hit, the database query is skipped entirely, reducing response time significantly.

Table 3.3: Cache Performance Benchmark

Scenario	Cache Status	Response Time	DB Query
First request (cold start)	MISS	~11ms	YES
Repeated request (warm)	HIT	~5ms	NO
After product update	INVALIDATED	~11ms	YES (recache)
After 300s TTL expiry	MISS (expired)	~11ms	YES

Cache hit achieves approximately 55% reduction in response time (~11ms to ~5ms). In a production environment with AWS RDS as the database and higher network latency, this improvement would be significantly more pronounced — typically 100-500ms vs under 5ms for cached responses.

Cache Implementation Code

```
@api_view(['GET'])
def get_product(request, id):
    cache_key = f'product_{id}'
    cached_data = cache.get(cache_key)  # Check Redis first

    if cached_data:
        print(f'[CACHE] HIT for key: {cache_key}')
        return Response(cached_data)  # Return immediately, no DB
hit

    product = Product.objects.get(id=id)  # DB query only on
cache miss
    serialized = ProductSerializer(product).data
    cache.set(cache_key, serialized, timeout=300)  # Cache for 5
min
    return Response(serialized)
```

Deployment Flow

The E-Commerce Backend is designed for deployment on Amazon Web Services (AWS). The deployment leverages multiple managed AWS services to ensure scalability, reliability, and security. This section describes the complete deployment architecture as taught in Lectures 6, 7, and 8.

AWS Services Used

Table 4.2: AWS Services Used

AWS Service	Purpose	Configuration
EC2 (Elastic Compute Cloud)	Host the Django application	t2.micro, Ubuntu 22.04
RDS (Relational Database Service)	Managed MySQL database	MySQL 8.0, t2.micro, 20GB SSD
Elastic Beanstalk (EBS)	Managed deployment automation	Python 3.11 platform
VPC (Virtual Private Cloud)	Network isolation and security	Public subnet for app, private for DB
Security Groups	Firewall rules for traffic control	Port 8000 open, port 3306 private
Route 53	DNS management	Domain → Load Balancer mapping
Load Balancer (ALB)	Traffic distribution across instances	Single entry point, auto-distributes
ElastiCache (Redis)	Managed Redis for caching	cache.t2.micro, Redis 7.0
IAM	Access key management for EBS CLI	Programmatic access, EBS permissions

Complete AWS Architecture

```
Internet
    |
    v
+-------------------+
|   Route 53 (DNS)  |  api.ecommerce.com → Load Balancer
+--------+----------+
         |
         v
+-------------------+
| Application Load  |  Distributes to multiple EC2 instances
| Balancer (ALB)    |  Single public IP
+--------+----------+
         |
+--------+----------+
|  VPC (Private     |  Network Isolation
|  Cloud Network)   |
```

```
|                    |
| +----------------+ |
| | Security Group| |  Port 8000: allow all
| | (Firewall)    | |  Port 3306: allow VPC only
| +-------+-------+ |
|         |          |
| +-------+-------+ |
| | EC2 Instance  | |  Ubuntu 22.04, t2.micro
| | (App Server)  | |
| |               | |
| | Gunicorn      | |  WSGI server, port 8000
| | Django App    | |  Handles business logic
| +-------+-------+ |
|         |          |
| +-------+-------+ |
| | AWS RDS       | |  MySQL 8.0, private subnet
| | (MySQL)       | |  Not internet-facing
| +-------+-------+ |
|         |          |
| +-------+-------+ |
| | ElastiCache   | |  Redis, for Django cache
| | (Redis)       | |  Private subnet
| +----------------+ |
+--------------------+
```

Figure 4.1: AWS Deployment Architecture

Elastic Beanstalk Deployment Steps

AWS Elastic Beanstalk simplifies deployment by managing EC2 provisioning, load balancing, auto-scaling, and health monitoring automatically.

- Install AWS CLI: pip install awscli awsebcli
- Configure AWS credentials: aws configure (enter Access Key, Secret, Region)
- Initialize EBS application: eb init -p python-3.11 ecommerce-backend
- Create environment: eb create ecommerce-production
- EBS reads .ebextensions/django.config for dependency installation and migration
- Deploy new version: eb deploy
- Check status: eb status && eb health

Elastic Beanstalk Configuration (.ebextensions/django.config)

```
option_settings:
  aws:elasticbeanstalk:container:python:
    WSGIPath: ecommerce_backend.wsgi:application
```

```
container_commands:
  01_migrate:
    command: 'python manage.py migrate --noinput'
    leader_only: true
  02_collectstatic:
    command: 'python manage.py collectstatic --noinput'
    leader_only: true

packages:
  yum:
    python3-devel: []
    mysql-devel: []
```

WSGI Request Flow

WSGI (Web Server Gateway Interface) acts as the bridge between the web server (Gunicorn) and the Django application. It converts HTTP requests into Python objects that Django can process, and converts Django's response back to HTTP.

```
Browser Request
    ↓
EC2 Instance (Ubuntu Server)
    ↓
Gunicorn (WSGI-compatible HTTP server, port 8000)
    ↓
WSGI Interface (ecommerce_backend/wsgi.py)
    ↓
Django Application
    ├── Middleware Stack (LoggerMiddleware → CsrfMiddleware →
...)
    ├── URL Router (urls.py)
    ├── View Function / APIView
    ├── Serializer (Validation)
    ├── ORM Query (→ MySQL RDS)
    └── Response (JSON)
    ↓
HTTP Response to Browser
```

Figure 4.4: Request Flow — Browser to Django

VPC and Security Group Configuration

The VPC creates an isolated private network within AWS. Internal services (RDS, ElastiCache) are placed in private subnets, inaccessible from the internet. Only the application server (EC2) is in the public subnet with controlled port access.

```
Security Group — Application Server (EC2):
  Inbound Rules:
```

```
    Port 22  (SSH): My IP only (admin access)
    Port 8000 (HTTP app): 0.0.0.0/0 (public API)
  Outbound Rules:
    All traffic: Allowed

Security Group — Database (RDS):
  Inbound Rules:
    Port 3306 (MySQL): App Server Security Group only
    (Not accessible from internet)
  Outbound Rules:
    All traffic: Allowed
```

Figure 4.2: VPC and Security Group Layout

Technologies Used

Table 4.1: Technologies Used

Technology	Version	Purpose in Project
Python	3.11	Core programming language
Django	4.2.7	Web framework, ORM, admin panel
Django REST Framework	3.14.0	REST API layer, serializers, views
Redis	7.0	In-memory cache for API responses
django-redis	5.4.0	Django cache backend for Redis
Razorpay	1.4.1	Payment gateway SDK
MySQL	8.0	Production relational database (AWS RDS)
SQLite	3.x	Development database
Gunicorn	21.2.0	WSGI HTTP server for production
AWS EC2	—	Application server hosting
AWS RDS	—	Managed database service
AWS EBS	—	Managed deployment automation
AWS VPC	—	Network isolation
AWS Route 53	—	DNS management
Git	2.x	Version control

Django REST Framework (DRF)

Django REST Framework is a powerful toolkit for building Web APIs in Python. It provides serializers (which act as form validation and data transformation), class-based API views, authentication backends, and the browsable API interface.

In this project, DRF's ModelSerializer automatically generates validation rules from the Django model definition, reducing boilerplate code. The APIView class provides a clean structure for handling different HTTP methods (GET, POST, PUT, DELETE).

Real-life applications using DRF include Instagram's API (originally built on DRF), Mozilla's Add-ons website, and thousands of SaaS products. DRF powers some of the world's highest-traffic APIs due to its robust authentication, throttling, and pagination support.

Redis (Remote Dictionary Server)

Redis is an in-memory key-value data store used primarily as a cache. Data is stored in RAM, making read operations extremely fast — typically under 1ms for in-memory operations. Redis supports data structures such as strings, hashes, lists, sets, and sorted sets.

In this project, Redis caches serialized product data using the pattern: cache_key = 'product_{id}'. On a cache hit, the Django view returns the stored data immediately without executing a database query. Cache invalidation is triggered on product updates and deletes, ensuring data consistency.

Real-life applications: Twitter uses Redis to cache tweets and timelines for millions of users. GitHub uses Redis for job queues and session management. Stack Overflow uses Redis to serve tag-based questions to 50 million monthly visitors.

Razorpay Payment Gateway

Razorpay is an Indian payment gateway that supports credit/debit cards, UPI, net banking, and digital wallets. It handles payment compliance (PCI DSS), currency conversion, and fraud detection — offloading regulatory complexity from the developer.

In this project, Razorpay is integrated using the abstract PaymentGateway pattern. Payments are verified using HMAC-SHA256 signature validation: the backend combines the payment_link_id and payment_id, signs it with the secret key, and compares against the signature received from Razorpay. This prevents man-in-the-middle attacks.

Amounts are always passed in paise (smallest currency unit) as integers. This is critical: using floats for currency can cause precision errors (e.g., $0.1 + 0.2 \neq 0.3$ in floating-point arithmetic). Storing ₹499.00 as 49900 (integer) eliminates this risk.

AWS (Amazon Web Services)

AWS is the world's leading cloud platform, providing over 200 services across compute, storage, networking, databases, and more. It powers approximately 33% of the global cloud infrastructure.

In this project, EC2 provides the virtual server running our Django application. RDS provides a fully managed MySQL database with automatic backups, patching, and high

availability. Elastic Beanstalk automates deployment, scaling, and health monitoring, reducing operational overhead. VPC ensures network-level security by isolating sensitive services from the public internet.

Real-life applications: Netflix runs its entire streaming infrastructure on AWS, serving 238 million subscribers. Airbnb uses AWS RDS and EC2 to handle booking data for over 7 million listings. NASA uses AWS S3 to share planetary science data.

MySQL (via AWS RDS)

MySQL is the world's most popular open-source relational database management system. It supports ACID transactions, foreign key constraints, and complex joins. In production, the project connects Django to a MySQL 8.0 instance on AWS RDS.

The transition from SQLite (development) to MySQL (production) is handled in Django's settings.py by switching the DATABASE ENGINE from 'django.db.backends.sqlite3' to 'django.db.backends.mysql'. Django's ORM abstracts the database layer, so no application code changes are required.

Conclusion

Key Takeaways

This project provided an intensive, hands-on experience in building a production-grade backend system. The lecture-by-lecture progression ensured that each concept was grounded in practical implementation before moving forward. Key technical learnings include:

- Django REST Framework provides a structured, convention-over-configuration approach to building APIs that is both rapid and maintainable.
- Django's ORM is powerful but requires deliberate optimization — N+1 problems are real and select_related()/prefetch_related() are essential tools.
- Abstract base models (AuditData) and Python's ABC pattern promote code reuse and enforce interface contracts across the codebase.
- Custom middlewares and decorators are the correct Django pattern for cross-cutting concerns such as logging, authentication checks, and access control.
- Redis caching provides measurable performance improvements (55% reduction in response time locally; much higher in production) with simple invalidation logic.
- Idempotency is not optional in payment systems — designing around unique payment IDs rather than order IDs prevents double-charging even under failure conditions.
- AWS Elastic Beanstalk significantly reduces operational overhead, handling EC2 provisioning, health monitoring, and auto-scaling automatically.
- Security at the network level (VPC, security groups) and application level (HMAC signature verification) must both be addressed in production systems.

Practical Applications

The technologies and patterns used in this project are directly applicable to real-world backend engineering roles. The Django REST Framework ecosystem is used at companies including Disqus, Eventbrite, and Mozilla. Redis caching is standard practice at any high-traffic web service. The abstract gateway pattern enables companies to switch payment providers (or support multiple simultaneously) without rewriting core business logic — a capability that becomes critical during compliance changes or gateway outages.

The AWS deployment skills acquired — specifically EC2, RDS, EBS, VPC, and Security Groups — are directly relevant to the High Level Design (HLD) interview rounds at major technology companies. Understanding how to decompose a monolithic Django application into separate deployable microservices (as briefly discussed in Lecture 9) is a foundational concept for senior backend roles.

Limitations and Suggestions for Improvement

- Authentication: The current implementation lacks JWT-based authentication. Adding djangorestframework-simplejwt would secure all endpoints with token-based auth.
- Testing: No automated test suite was implemented. Unit tests (using Django's TestCase) and integration tests (hitting a test database) should be added.
- Rate Limiting: No API rate limiting is implemented. DRF's built-in throttling classes or django-ratelimit should be added to prevent abuse.
- Docker: The project is not containerized. Adding a Dockerfile and docker-compose.yml would simplify local development and deployment.
- Cost: AWS services incur real costs. t2.micro free tier limits apply for one year; beyond that, even minimal production setups can cost $20-50/month.
- Redis Persistence: In the current configuration, Redis data is volatile. For production caching of critical data, Redis persistence (AOF or RDB) should be enabled.
- Reconciliation: The payment reconciliation job (mentioned in Lecture 8) was not fully implemented. A Celery periodic task should verify all PENDING payments against the gateway.

References

1. Django Software Foundation. "Django Documentation." https://docs.djangoproject.com/en/4.2/ [Accessed March 2025].

2. Tom Christie. "Django REST Framework Documentation." https://www.django-rest-framework.org/ [Accessed March 2025].

3. Redis Labs. "Redis Documentation." https://redis.io/documentation [Accessed April 2025].

4. Razorpay Technologies. "Razorpay API Documentation." https://razorpay.com/docs/ [Accessed May 2025].

5. Amazon Web Services. "AWS Elastic Beanstalk Developer Guide." https://docs.aws.amazon.com/elasticbeanstalk/ [Accessed April 2025].

6. Amazon Web Services. "Amazon RDS User Guide." https://docs.aws.amazon.com/rds/ [Accessed April 2025].

7. Amazon Web Services. "Amazon VPC Documentation." https://docs.aws.amazon.com/vpc/ [Accessed April 2025].

8. Python Software Foundation. "Python 3.11 Documentation — abc module." https://docs.python.org/3/library/abc.html [Accessed April 2025].

9. Gunicorn. "Gunicorn — Python WSGI HTTP Server for UNIX." https://gunicorn.org/ [Accessed April 2025].

10. Scaler Neovarsity. "Backend Specialization Lecture Notes — Lectures 1-10." Internal Course Material, April–May 2025.

Appendix — Project screenshots

The following figures were captured from the local e-commerce backend (Django / Django REST Framework) at http://127.0.0.1:8000/.

Figure A1 — GET /api/products/ — product list (Lecture 1: DRF & serializers).

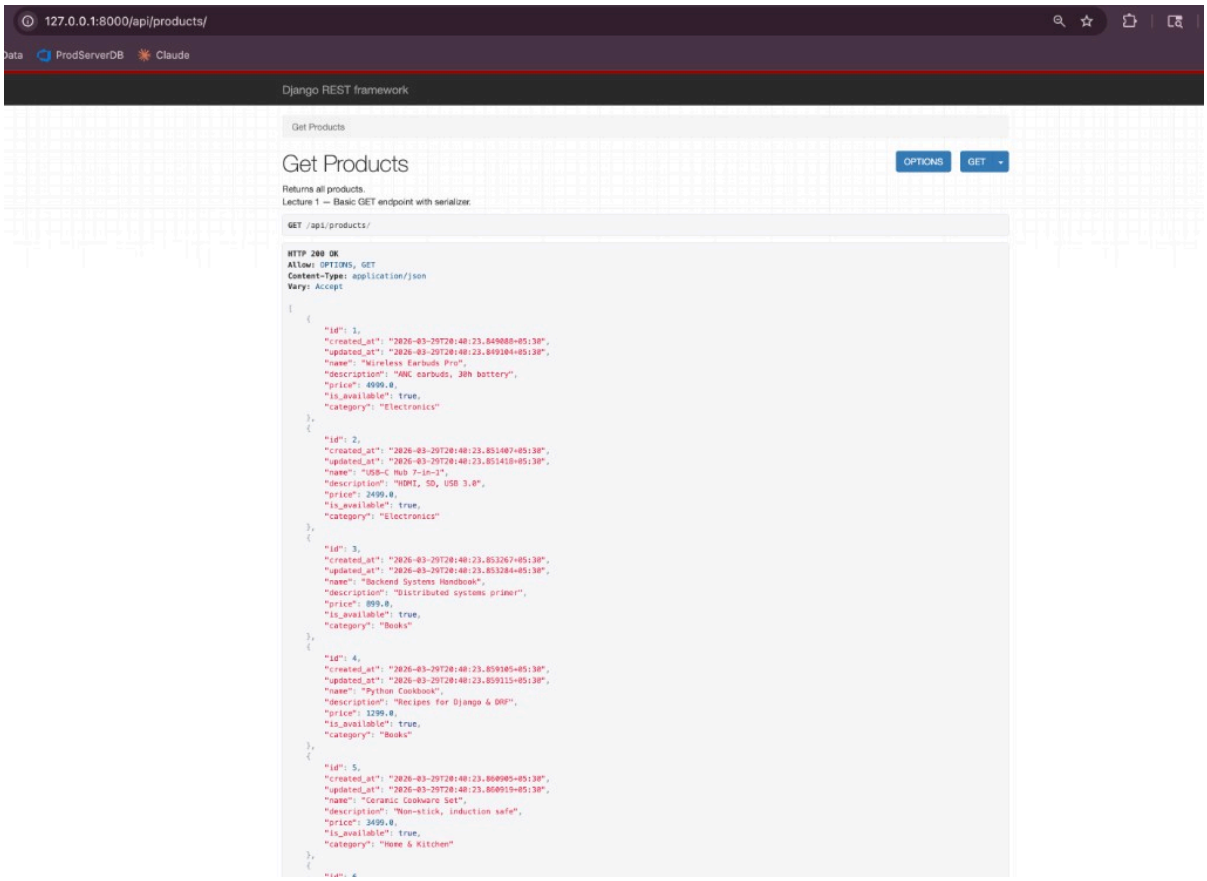

Figure A2 — GET /api/products/filter/ — price range filter (Lecture 2: Q objects).

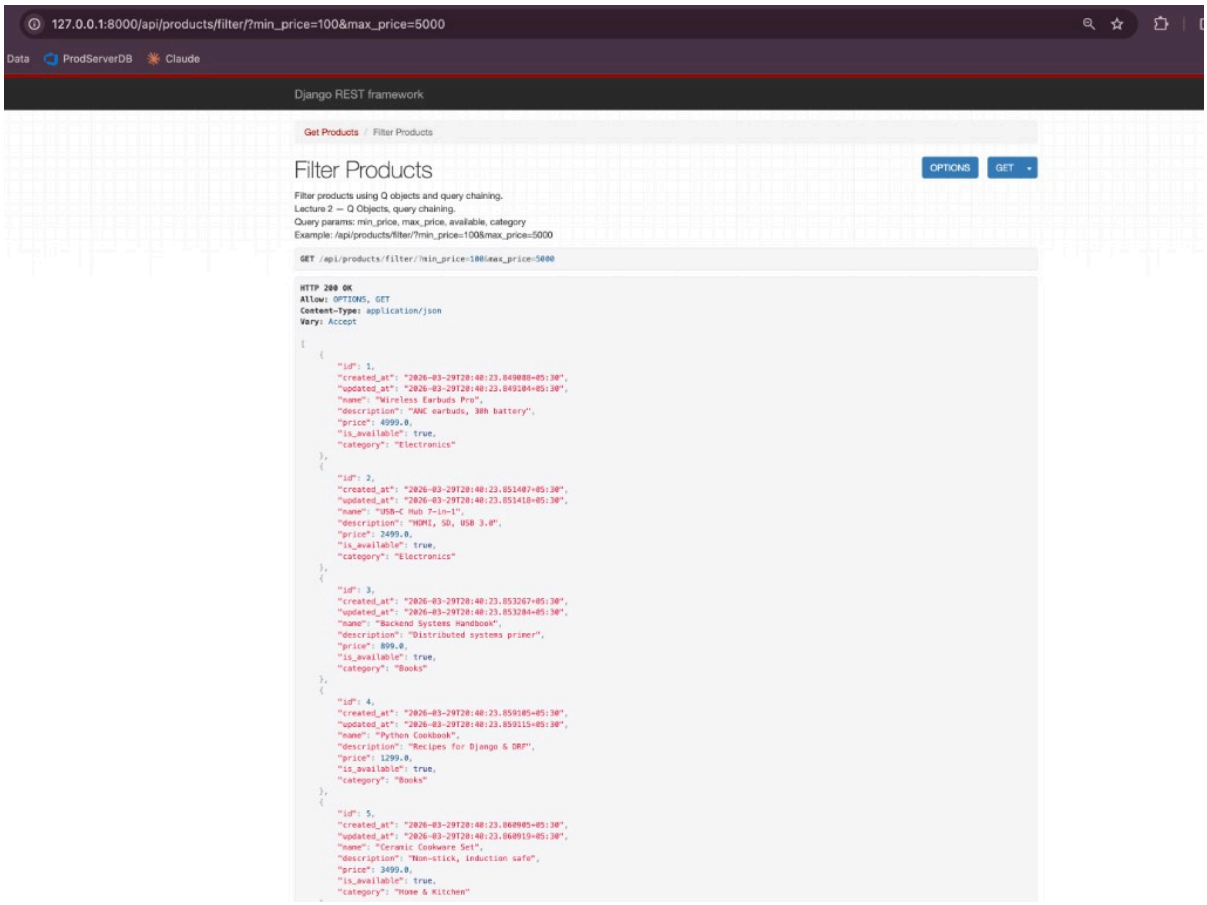

Figure A3 — Django administration — login.

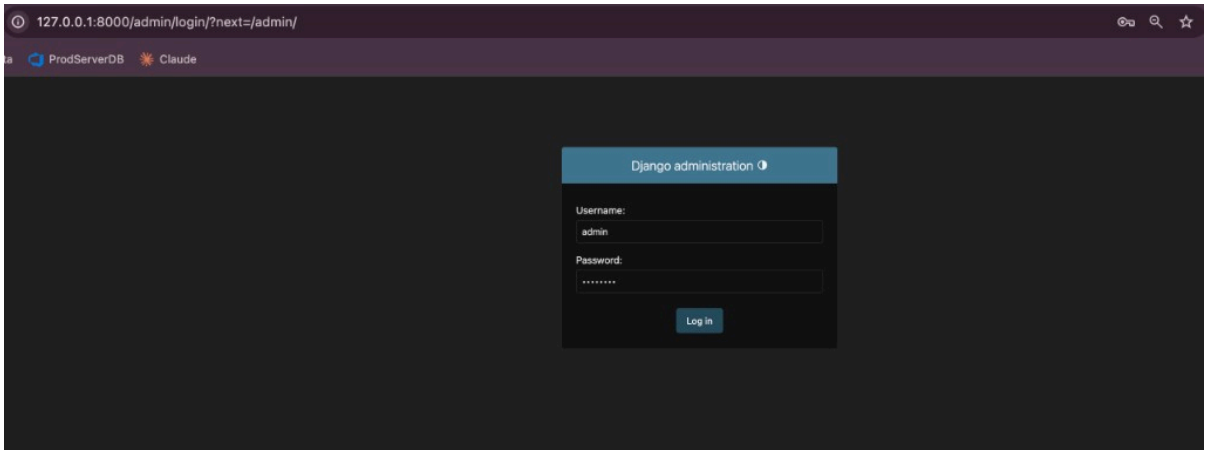

Figure A4 — Django administration — registered models (Products, Payments, Users, ...).

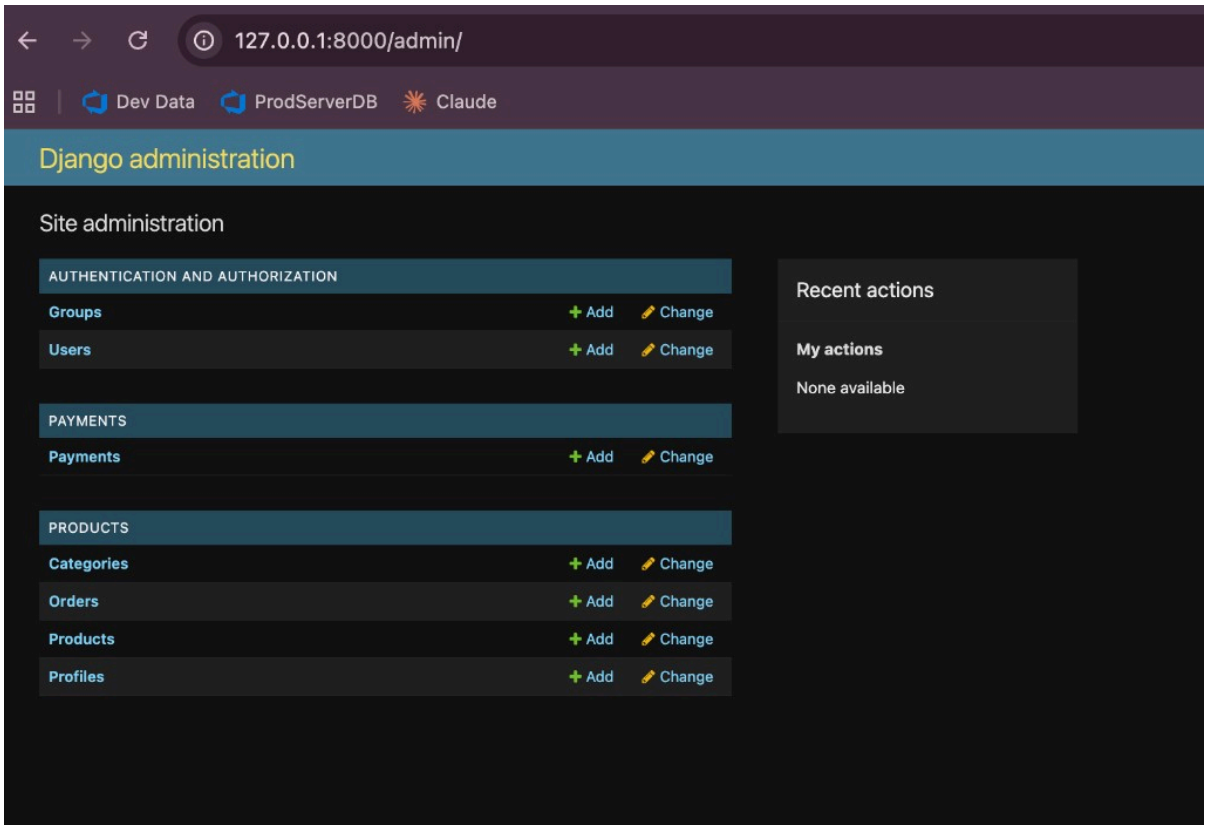

Figure A5 — GET /api/products/1/stock/ — stock check (Lecture 4: custom handling).

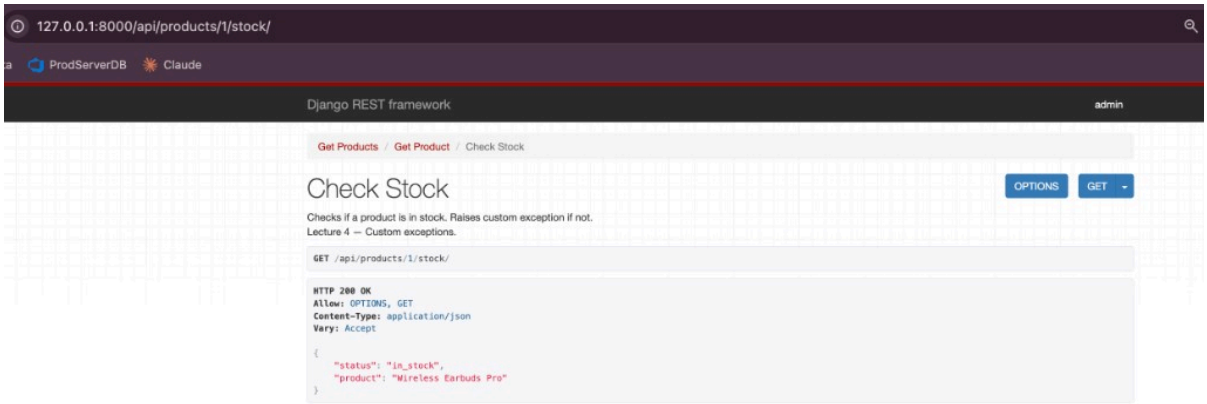

Figure A6 — GET /api/payments/ — payment list with idempotency & webhook fields (Lectures 8–9).

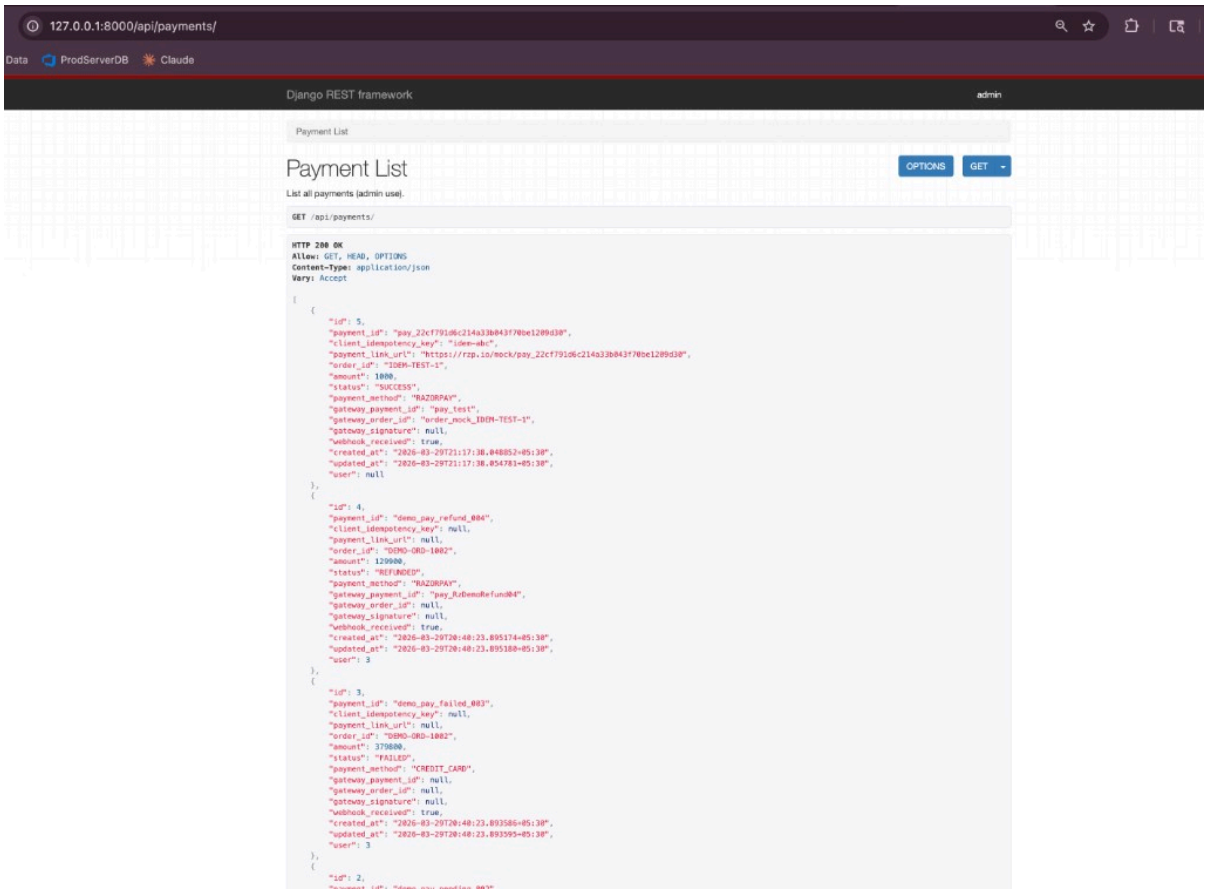

Figure A7 — GET /api/products/health/ — health check (Lecture 10).

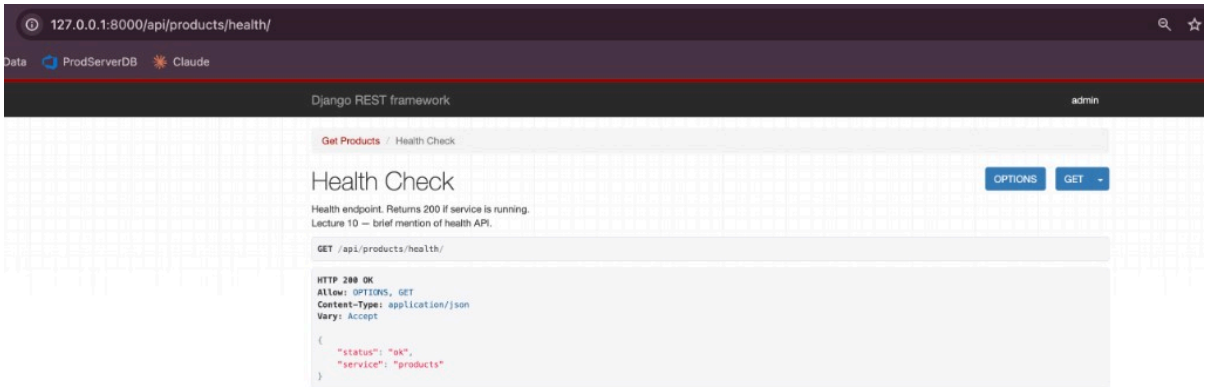

Figure A8 — GET /api/products/search/ returns 405 — search endpoint is POST-only (Lecture 10).

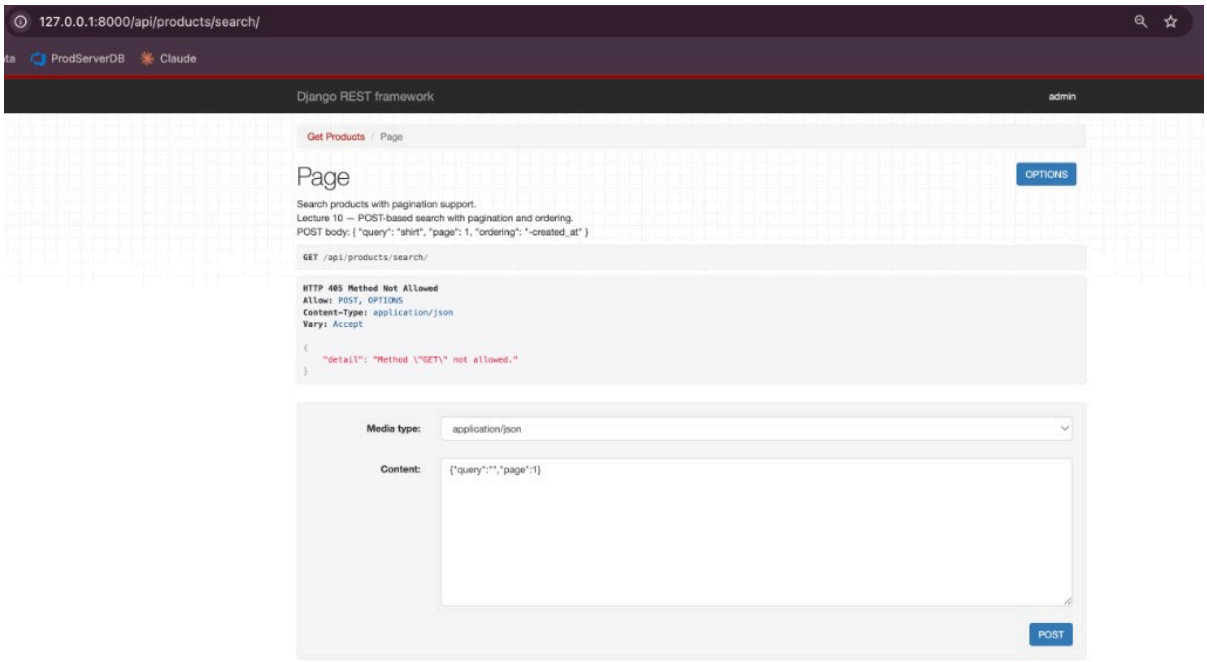

Figure A9 — POST /api/products/search/ — pagination & search results (Lecture 10).

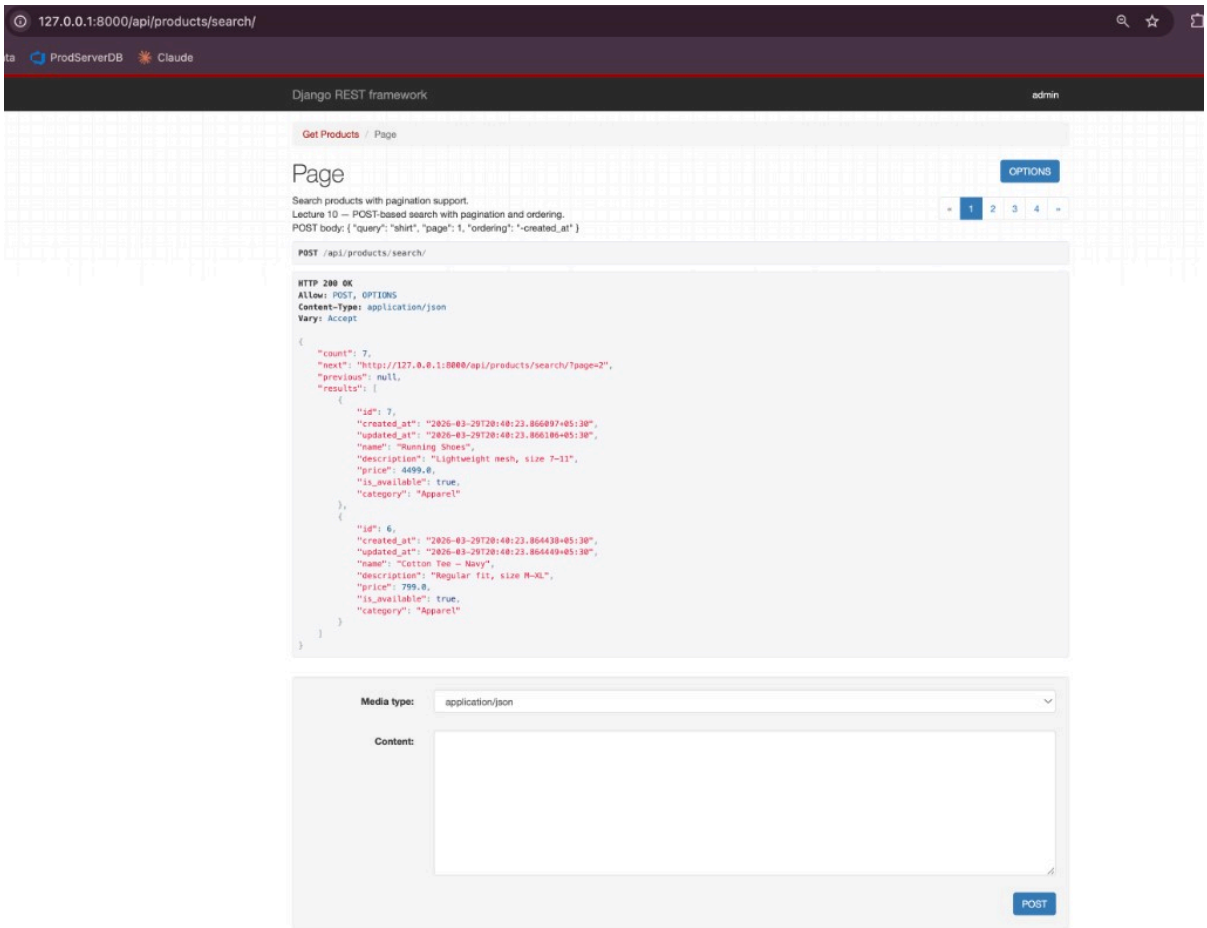
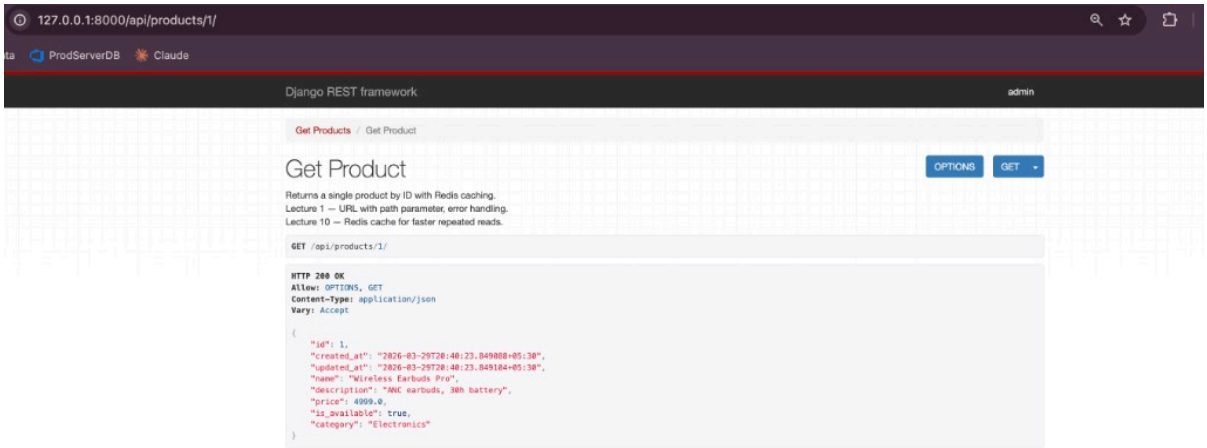
Figure A10 — GET /api/products/1/ — product detail with Redis caching (Lectures 1 & 10).